49. Group Anagrams

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

> **Input:** `strs = ["eat","tea","tan","ate","nat","bat"]`
>
> **Output:** `[["bat"],["nat","tan"],["ate","eat","tea"]]`
>
> **Explanation:**
>
> - There is no string in strs that can be rearranged to form `"bat"`.
> - The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
> - The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

> **Input:** `strs = [""]`
>
> **Output:** `[[""]]`

Example 3:

> **Input:** `strs = ["a"]`
>
> **Output:** `[["a"]]`

Constraints:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

DS / Day 97 / 13-12-26

Leet code 49 - Group Anagrams.

You are given an array of strings called strs.

Your task is to group all strings that are anagrams of each other into separate groups.

What is an anagram?

→ Two strings are anagrams if:
- they contain exactly the same characters.
- each char appears the same no. of times
- the order of chars can be diff

eg input: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

expected output:-

[["bat"], ["nat", "tan"], ["ate", "tea", "eat"]]

The order of groups doesn't matter.

What's the actual challenge?

↳ The challenge is not simply identifying whether two strings are anagrams.

It is finding an efficient way to identify which strings belong to the same group.

think about this:-

eat, tea & ate have diff. arrangements, but if we sort them:-

eat → aet

tea → aet

ate → aet

they all become identical.

Approach 1: sorting + Hash-map ⇒ we need a common identifier for all anagrams, since anagrams contain exactly the same characters, sorting them alphabetically produces the same string.

for ex	original string	after sorting
	eat	aet
	tea	aet
	tan	ant
	ate	aet
	nat	ant
	bat	abt.

→ so, all strings with ex "aet" belong to one group

→ all string " " "ant" " " "

→ " " " " "abt" " " "

Ø

sort(key.begin(), key.end()); → we sort the characters alphanumerically.

mp[key].push_back(s); for ex → s = "tea" key = "aet".

↳ we use the sorted string as the key & insert the original string into its corresponding group.

If the key doesn't already exist, the hash map automatically creates an empty vector for it.

→ we create our final answer. it is a 2D vector bcz each group contains multiple strings.

vector<vector<string>> ans;

for (auto it : mp) { → we iterate through every key-value pair in the hash-map.

ans.push_back(it.second); ↳ represents the vector of original strings.

} we add each group to our final answer.

return ans; ↳ returning all the anagram groups

T.C = O(n x klog k)

S.C = O(n x k) ← we store the original strings in the hashmap & the returning groups in the answer. The sorted keys also require space.

n = no. of strings.

k = max. length of a string.

↳ copying the string takes O(k).

↳ sorting " " " O(k log k).

↳ inserting it into the hash map takes O(k) expected time for hashing & storage.

we can use an unordered_map to store these groups,
why a hash map? we need to map: ___/___/___

sorted string → All original strings having that sorted string.
for ex- act → ["cat", "tac", "atc"]
ant → ["tan", "nat"]
bat → ["bat"]
the key identifies the group & the value stores the original strings,

2d array/vector class Solution { public:
containing groups of strings. vector<vector<string>> groupAnagrams(vector<string>& strs) {

unordered_map<string, vector<string>> mp;
↳ we create a hash map.
- string represents the key (sorted string)
- vector<string> stores all original strings belonging to that key.

for ex-
mp["act"] = {"cat", "tac", "atc"}.

for (string s : strs) { → we iterate through every string in the input array.

string key = s; → we create a copy of the original string, why? bcz we want to sort the copy without changing the original string.

dry run →

strs = {"eat", "tea", "tan", "ate", "nat", "bat"};

initially, mp = {}.

item 1 → eat, sorted key: aet
mp["aet"] = ["eat"]

item 2 → tea, sorted key: aet
mp["aet"] = ["eat", "tea"]

item 3 → tan, sorted key = ant
mp["ant"] = ["tan"].

item 4 → ate, sorted key: aet
mp["aet"] = ["eat", "tea", "ate"]

item 5 → nat, sorted key = ant
mp["ant"] = ["tan", "nat"]

item 6 → bat, sorted key: abt
mp["abt"] = ["bat"].

We extract the vectors from the map & return:-

[["eat", "tea", "ate"],
["tan", "nat"],
["bat"]].

Remember the o/p order can be diff. bcuz unordered_map doesn't maintain insertion order.

Accepted 130 / 130 testcases passed

Akankshyaa submitted at Nov 13, 2026 13:48

Analysis | Solution

Save 41%

Big Diwali Dhamaka

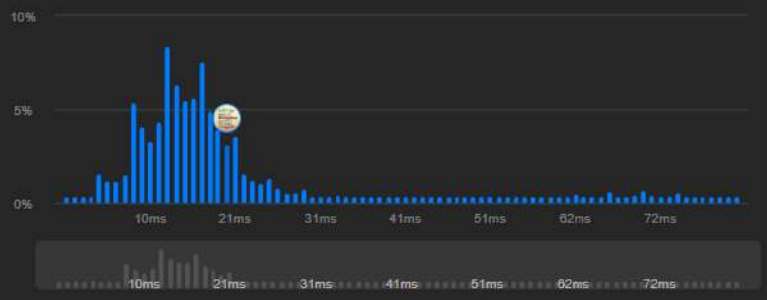
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

20 ms | Beats 42.65%

Memory

26.22 MB | Beats 30.92%



Code | C++

```cpp
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string,vector<string>>mp;
        for (string s: strs){
            string key=s;
            sort(key.begin(),key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>> ans;
        for(auto it: mp){
            ans.push_back(it.second);
        }
        return ans;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

strs =

["eat","tea","tan","ate","nat","bat"]

Output